Smart Contract Assessment

Moonwell

HALBORN

Summary

0% OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

ABOUT THIS REPORT

ASSESSMENT TYPE
Assurance Assessment

DATE OF ENGAGEMENT
Oct 15, 2026 - Oct 19, 2026

SEVERITY BREAKDOWN

6	6	0	0
All Findings	Pending	Solved	Risk Accepted

Critical	0	
High	0	
Medium	0	
Low	1	Pending 1
Informational	5	Pending 5

INTRODUCTION

`Moonwell` engaged `Halborn` to perform a security assessment of their smart contracts from October 16th, 2026 to October 19th, 2026. The assessment scope was limited to the smart contracts provided to Halborn. Commit hashes and additional details are available in the Scope section of this report.

The `Moonwell` codebase in scope consists of changes made to the mToken markets that replace balance based cash with an internally tracked internalCash value, so direct token transfers no longer move the exchange rate or count toward supply caps, together with admin functions to sweep excess cash and lower tracked cash after external losses, matching updates to the MWethOwnerWrapper, and the MIP-X71 proposal that upgrades the markets on Ethereum, Base, and OP Mainnet.

ASSESSMENT SUMMARY

`Halborn` was allocated 2 days for this engagement and assigned 1 full-time security engineer to conduct a comprehensive review of the smart contracts within scope. The engineer is an expert in

blockchain and smart contract security, with advanced skills in penetration testing and smart contract exploitation, as well as extensive knowledge of multiple blockchain protocols.

The objectives of this assessment are to:

- Identify potential security vulnerabilities within the smart contracts.
- Verify that the smart contract functionality operates as intended.

In summary, `Halborn` identified several areas for improvement to reduce the likelihood and impact of security risks, which should be addressed by the `Moonwell` team. The main recommendations are:

- `Make MErc20Delegate.getCashPrior() return the lower of internalCash and the market's underlying balance, so external losses and fees charged to the market on payouts are priced immediately instead of waiting for a governance call to _syncCashDown().`
- `Add a final action to each chain's MIP-X71 batch that reverts unless every market returned by Comptroller.getAllMarkets() uses the new implementation, so a market listed after the proposal is built cannot keep the previous cash accounting.`
- `Implement a two step ownership transfer and disable renounceOwnership() in MWethOwnerWrapper, declaring the pending owner after the existing state variables so the proxy storage layout is preserved.`

TEST APPROACH AND METHODOLOGY

`Halborn` conducted a combination of manual code review and automated security testing to balance efficiency, timeliness, practicality, and accuracy within the scope of this assessment. While manual testing is crucial for identifying flaws in logic, processes, and implementation, automated testing enhances coverage of smart contracts and quickly detects deviations from established security best practices.
The following phases and associated tools were employed throughout the term of the assessment:

- Research into the platform's architecture, purpose and use.
- Manual code review and walkthrough of smart contracts to identify any logical issues.
- Comprehensive assessment of the safety and usage of critical Solidity variables and functions within scope that could lead to arithmetic-related vulnerabilities.
- Local testing using custom scripts (`Foundry`).

- Fork testing against main networks (`Foundry`).
- Static security analysis of scoped contracts, and imported functions (`Slither`).

Risk Methodology

Every vulnerability and issue observed by Halborn is ranked based on **two sets** of **Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

4.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker's control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (m_e)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A)	1
	Specific (AO:S)	0.2
Attack Cost (AC)	Low (AC:L)	1
	Medium (AC:M)	0.67
	High (AC:H)	0.33
Attack Complexity (AX)	Low (AX:L)	1
	Medium (AX:M)	0.67
	High (AX:H)	0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

4.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

IMPACT METRIC (m_I)	METRIC VALUE	NUMERICAL VALUE
Confidentiality (C)	None (C:N)	0
	Low (C:L)	0.25
	Medium (C:M)	0.5
	High (C:H)	0.75
	Critical (C:C)	1
Integrity (I)	None (I:N)	0
	Low (I:L)	0.25
	Medium (I:M)	0.5
	High (I:H)	0.75
	Critical (I:C)	1
Availability (A)	None (A:N)	0
	Low (A:L)	0.25
	Medium (A:M)	0.5
	High (A:H)	0.75
	Critical (A:C)	1
Deposit (D)	None (D:N)	0
	Low (D:L)	0.25
	Medium (D:M)	0.5
	High (D:H)	0.75
	Critical (D:C)	1
Yield (Y)	None (Y:N)	0
	Low (Y:L)	0.25
	Medium (Y:M)	0.5
	High (Y:H)	0.75

IMPACT METRIC (m_I)	METRIC VALUE	NUMERICAL VALUE
	Critical (Y:C)	1

Impact I is calculated using the following formula:

$$I = max(m_I) + \frac{\sum m_I - max(m_I)}{4}$$

4.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

SEVERITY COEFFICIENT (C)	COEFFICIENT VALUE	NUMERICAL VALUE
Reversibility (r)	None (R:N)	1
	Partial (R:P)	0.5
	Full (R:F)	0.25
Scope (s)	Changed (S:C)	1.25
	Unchanged (S:U)	1

Severity Coefficient C is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score S is obtained by:

$$S = min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

Critical	High	Medium	Low	Informational

9 - 10	7 - 8.9	4.5 - 6.9	2 - 4.4	0 - 1.9

Scope

URL

(a) URL:
https://github.com/moonwell-fi/moonwell-contracts-v2/pull/696/changes/BASE..05388dd69e296db625900b735127d6389250520d

(b) Items in scope:

- proposals/mips/mip-x71 1 file
  - mip-x71.sol Solidity
- src 4 files
  - 4626 1 file
    - LibCompound.sol Solidity
  - MErc20Delegate.sol Solidity
  - MWethDelegate.sol Solidity
  - MWethOwnerWrapper.sol Solidity

Out-of-Scope: Third party dependencies and economic attacks.

Out-of-Scope: New features/implementations after the remediation commit IDs.

Assessment Summary & Findings Overview

#	TITLE	SEVERITY	SCORE	STATUS
FIND-003	External losses are not reflected in the exchange rate until governance syncs cash	Low	2.1	Pending
FIND-005	Market list is fixed at build time without an execution-time check	Informational	0.8	Pending
FIND-004	Single-step ownership transfer process	Informational	0.3	Pending

#	TITLE	SEVERITY	SCORE	STATUS
FIND-002	Market remains frozen after _syncCashDown removes nearly all suppliers' equity	Informational	0.2	Pending
FIND-006	Interest accrued at stale cash when re-upgrading after a rollback	Informational	0.1	Pending
FIND-001	Discrepancy between documentation and implementation	Informational	—	Pending

Findings & Tech Details

FIND-003 Pending

External losses are not reflected in the exchange rate until governance syncs cash

Low · 2.1 AO:A/AC:L/AX:H/C:N/I:N/A:N/D:M/Y:N/R:N/S:C

DESCRIPTION

After MIP-X71, `MErc20Delegate.getCashPrior()` returns the internally tracked `internalCash` instead of the market's underlying balance. `internalCash` only changes through `doTransferIn()`, `doTransferOut()`, `_becomeImplementation()` and `_syncCashDown()`. `_syncCashDown()` can only be called by the market admin, which is governance or, for the Base WETH market, `MWethOwnerWrapper` through its governance owner.

When the market loses underlying outside normal protocol operations, for example through an issuer seizure or burn, `internalCash` keeps its pre-loss value until a governance proposal executes `_syncCashDown()`. During that window:

- The exchange rate still reflects the pre-loss cash, so the first suppliers to redeem receive their full pre-loss value in real tokens.
- The redeem and borrow cash checks pass against `internalCash`, but once a payout exceeds the remaining real balance the underlying transfer in `doTransferOut()` reverts, so later suppliers can only withdraw what is left.
- After the sync, the remaining suppliers absorb the entire loss instead of a pro rata share.
- Collateral in the market is overvalued, so borrowers can borrow more than the real backing allows, and the interest rate model understates utilization and the borrow rate.

The same gap between `internalCash` and the real balance can also come from the underlying token itself during normal payouts. `doTransferIn()` credits `internalCash` with the amount actually received, but `doTransferOut()` debits the requested amount without measuring how much the market's balance fell. If the underlying token takes more than the transferred amount from the sender, each redeem, borrow, reserve reduction or excess sweep leaves `internalCash` above the balance, with the same consequences. None of the underlying tokens currently listed on Ethereum, Base or OP Mainnet is known to behave this way, but an upgradeable token could adopt such a fee.

Before MIP-X71, cash was read from the token balance, so the same loss was reflected in the exchange rate immediately and shared pro rata by all suppliers. The pause guardian can pause minting and borrowing on the market, but the Comptroller offers no way to pause redemptions, so the race

between suppliers cannot be stopped while the governance proposal is pending. The proposal documentation states that governance lowers accounted cash after such a loss, but does not describe this window or its effect on how the loss is distributed.

RECOMMENDATION

Make `MErc20Delegate.getCashPrior()` return the lower of `internalCash` and the market's underlying balance, without writing to storage, so external losses are priced immediately and shared pro rata by all suppliers. Additionally, call `_syncCashDown()` and schedule implementation upgrades, whose `_becomeImplementation()` always re-syncs `internalCash` to the underlying balance, only once a loss is known to be permanent. Document in the proposal description the loss window, the response procedure, and that direct transfers made while the balance is below `internalCash` raise the exchange rate up to `internalCash`.

FIND-005 Pending

Market list is fixed at build time without an execution-time check

Informational · 0.8 AO:A/AC:L/AX:H/C:N/I:N/A:N/D:M/Y:N/R:P/S:U

DESCRIPTION

`mipx71._buildUpgrades()` reads `Comptroller.getAllMarkets()` on each chain's fork when the proposal is built, and pushes one `_setImplementation(newImplementation, true, "")` action for every market it finds (for Base `MOONWELL_WETH`, through `MWETH_OWNER_WRAPPER` after the wrapper upgrade). The batch contains no action that checks the market list or each market's implementation when it executes on-chain; the implementation checks and `validate()` only run in the script against the local fork.

The proposal documentation states that the canonical implementation addresses in the repository are updated after execution so that future markets use the new accounting, and the market listing templates (`MarketAdd`, `MarketAddV2` and `MarketAddV3`) deploy new markets with the `MTOKEN_IMPLEMENTATION` registry entry. A market deployed from the registry before that update and listed by governance after MIP-X71 is built, for example during its voting or timelock period, therefore runs the previous implementation and is not part of the MIP-X71 batch. After execution, that market still reads cash from its token balance, so direct transfers raise its exchange rate and count toward its supply cap, while every other market follows the internal cash accounting. The documentation covers the upgrade of all current markets and the registry update for future markets, but not this interval.

RECOMMENDATION

Add a final action to each chain's batch that calls a stateless view helper, similar to `MarketAddChecker`, which reverts unless every market returned by `Comptroller.getAllMarkets()` uses the new implementation (`MWETH_IMPLEMENTATION` for `MOONWELL_WETH`). Additionally, update `chains/*.json` as soon as the implementations are deployed, pointing `MTOKEN_IMPLEMENTATION` and `MWETH_IMPLEMENTATION` at the new implementations and recording the previous ones under their `_DEPRECATED_V1` names, so that markets deployed afterwards use the new implementation. Make `mipx71._buildUpgrades()` skip markets already on the new implementation, while still rejecting any other unexpected implementation, so the proposal can still be rebuilt after such a market is listed. In `mipx71._validateMarkets()`, apply the check that `internalCash` equals the underlying balance only to the markets the batch upgrades, and require only that `internalCash` is at most the underlying balance for skipped markets.

FIND-004 Pending

Single-step ownership transfer process

Informational · 0.3 AO:S/AC:L/AX:L/C:N/I:N/A:M/D:N/Y:N/R:F/S:C

DESCRIPTION

`MWethOwnerWrapper` inherits OpenZeppelin's `OwnableUpgradeable` to restrict all of its admin functions to the contract owner. After MIP-X71 the wrapper remains the admin of the Base WETH market, and every passthrough, including the new `_setImplementation()`, `_sweepExcessCash()` and `_syncCashDown()`, as well as `_setPendingAdmin()` and `withdrawToken()`, is restricted with `onlyOwner`.

The `Ownable` pattern lets the owner transfer ownership through `transferOwnership()`, which completes in a single step, so the address receiving ownership is never verified to be active and willing to assume ownership responsibilities. Additionally, `renounceOwnership()` is not overridden, so the owner can give up ownership entirely. If ownership is renounced or transferred to an inaccessible address, every admin function of the wrapper becomes unreachable, the Base WETH market admin cannot be moved because `_setPendingAdmin()` is also owner-only, and the WETH held by the wrapper cannot be withdrawn. Control can only be restored by having `TEMPORAL_GOVERNOR`, as owner of `MRD_PROXY_ADMIN`, upgrade the wrapper proxy to an implementation that rewrites the owner.

Instances of this issue can be found in:

- `MWethOwnerWrapper.transferOwnership()`
- `MWethOwnerWrapper.renounceOwnership()`

RECOMMENDATION

Implement a two step ownership transfer in `MWethOwnerWrapper`, where the new owner must accept ownership before it takes effect, and override `renounceOwnership()` so that it always reverts. Declare the pending owner as a new state variable after `mToken` and `weth`, keeping `OwnableUpgradeable` as the only ownership parent, so the storage layout of the deployed proxy stays unchanged.

FIND-002 Pending

Market remains frozen after _syncCashDown removes nearly all suppliers' equity

Informational · 0.2 AO:S/AC:L/AX:H/C:N/I:N/A:N/D:H/Y:N/R:F/S:C

DESCRIPTION

The `MErc20Delegate._syncCashDown()` function lets the market admin lower `internalCash` to the underlying balance after a loss outside the market, such as an issuer seizure or burn. MIP-X71 adds it together with internal cash accounting, where `getCashPrior()` returns `internalCash` instead of the token balance. The function only lowers `internalCash` and leaves `totalReserves` unchanged.

However, when governance syncs a loss that removes all or nearly all of the suppliers' equity (`internalCash + totalBorrows − totalReserves`), the market stops working. When `totalReserves` exceeds `internalCash + totalBorrows`, `exchangeRateStoredInternal()` returns a math error, so `exchangeRateStored()` reverts and `getAccountSnapshot()` fails. When they are equal, the exchange rate is zero. While borrows are outstanding, `accrueInterest()` reverts once the equity is zero or negative, because the interest rate model computes `cash + borrows − reserves`, which underflows when the equity is negative and is a division by zero when it is exactly zero. It also reverts while a small positive equity remains, because utilization pushes the borrow rate above `borrowRateMaxMantissa`. With a jump multiplier of 9e18 per year and a kink of 80%, this happens once borrows exceed about 18 times the remaining equity. As a result:

- Whenever `accrueInterest()` reverts as described above, or once `totalReserves` exceeds `internalCash + totalBorrows`, mint, redeem and borrow on the market fail, and ERC-4626 views built on `LibCompound.viewExchangeRate()` revert.
- Whenever `accrueInterest()` reverts, repay, liquidation, `_reduceReserves()`, `_setInterestRateModel()` and any further `_syncCashDown()` also revert.
- Once `totalReserves` exceeds `internalCash + totalBorrows`, the Comptroller liquidity check returns `SNAPSHOT_ERROR` for every account that entered the market, so these accounts can no longer borrow in any market, redeem from other markets they entered, or be liquidated.

Before MIP-X71 the same loss also broke the market, but it recovered as soon as the lost tokens were returned, because cash was read from the token balance. With internal cash accounting, returned tokens are treated as excess and the market stays frozen:

- `_syncCashDown()` never raises `internalCash`.
- `_setImplementation(newImplementation, true, "")`, the mode the repository requires for every upgrade, first sweeps the returned tokens to the admin through

`_resignImplementation()`.

- While borrows are outstanding and `internalCash` is non zero, `_becomeImplementation()` calls `accrueInterest()` before re-syncing, so both `_setImplementation()` with either `allowResign` value and a direct admin call to `_becomeImplementation()` revert.

Without outstanding borrows, the admin restores the market by calling `_becomeImplementation()` directly on the market, which re-syncs `internalCash` to the balance without sweeping. `MWethOwnerWrapper` has no passthrough for that call, so the simplest route for the Base WETH market is `MWethOwnerWrapper._setImplementation()` with `allowResign = false`, which departs from the repository upgrade rule. With outstanding borrows and a non zero `internalCash`, the existing implementations allow restoring the market by switching with `allowResign = false` to an implementation that reads cash from the token balance and does not accrue interest in `_becomeImplementation()`, such as the archived `MTOKEN_IMPLEMENTATION_DEPRECATED_V1` (or `MWETH_IMPLEMENTATION_DEPRECATED_V1` for WETH markets), accruing interest there, and switching back within the same block. Neither the contract NatSpec nor the proposal documentation describes this condition or the recovery procedure. Reaching the state requires reserves close to or above the outstanding borrows and an external loss that removes all or nearly all of the suppliers' equity, followed by a governance call to `_syncCashDown()`.

RECOMMENDATION

Make `MErc20Delegate._becomeImplementation()` skip `accrueInterest()` whenever accrual at the current cash would revert, namely when borrows are outstanding and `getCashPrior() + totalBorrows` is not greater than `totalReserves` (checked before calling the interest rate model) or when the model returns a borrow rate above `borrowRateMaxMantissa`. Once the lost tokens are returned, governance then restores the market by calling `_becomeImplementation()` directly on the market, or, for the Base WETH market, through `MWethOwnerWrapper._setImplementation()` with `allowResign = false`. Additionally, document this condition and the recovery procedure in the `_syncCashDown()` and `_becomeImplementation()` NatSpec, the proposal description and the repository upgrade rules, including the Base WETH exception to the `allowResign = true` rule.

FIND-006 Pending

Interest accrued at stale cash when re-upgrading after a rollback

Informational · 0.1 AO:S/AC:L/AX:H/C:N/I:N/A:L/D:N/Y:L/R:P/S:U

DESCRIPTION

The `MErc20Delegate._becomeImplementation()` function runs when a market switches to the internal cash implementation. It reads the current `internalCash` value and, if it is non-zero, treats the outgoing implementation as one that already maintained internal cash, calls `accrueInterest()` at that tracked cash, and then sets `internalCash` to the underlying balance of the market. When `internalCash` is zero, the accrual is skipped because the slot is considered unset, as during the first migration from the balance based implementation.

However, a non-zero `internalCash` does not prove that the outgoing implementation kept it current. The balance based implementations, such as the archived `MTOKEN_IMPLEMENTATION_DEPRECATED_V1` that MIP-X71 keeps in the address registry, never write the `internalCash` storage slot, and `MErc20Delegate._resignImplementation()` never clears it. If governance rolls a market back from the internal cash implementation to a balance based one, for example as an incident response, the slot keeps the value it had before the rollback while the real cash keeps changing. When governance later upgrades the market to the internal cash implementation again, `_becomeImplementation()` sees the stale non-zero value and calls `accrueInterest()` under the new code, where `getCashPrior()` returns that stale value. Interest for the whole period since the last accrual is therefore priced at a utilization derived from the stale cash instead of the real balance, and only afterwards is `internalCash` re-synced. The NatSpec states that `internalCash` is unset after a legacy implementation, which only holds for the first migration.

As a result, borrowers are charged the wrong amount of interest for the period between the last accrual and the re-upgrade. For a market with 160 tracked cash and 40 borrowed that receives 10,000 of additional supply under the rolled back implementation and then goes 365 days without an accrual, the re-upgrade charges borrowers 7.60 of interest instead of the 4.07 charged at the real cash, an overcharge of about 87% for that window. If the stale value is above the real cash, borrowers are undercharged instead. If total borrows are non zero and the stale cash plus total borrows is at or below total reserves, the interest rate model's `utilizationRate()` reverts, and a stale utilization high enough to push the borrow rate above `borrowRateMaxMantissa` reverts in `accrueInterest()`; in both cases the re-upgrade itself reverts. The mispriced window only covers the time since the last accrual, which is short on actively used markets.

RECOMMENDATION

Have every proposal that upgrades a market to the internal cash implementation while its `internalCash` storage slot is non zero, such as a re-upgrade after a rollback to a balance based implementation, call `accrueInterest()` on the market in the same transaction before `_setImplementation()`, so the accrual inside `MErc20Delegate._becomeImplementation()` has no elapsed period left to price at the stale `internalCash`. Additionally, document this requirement in the `_becomeImplementation()` NatSpec and the repository upgrade rules, and correct the NatSpec statement that `internalCash` is unset after a legacy implementation so it covers rollbacks.

FIND-001 Pending

Discrepancy between documentation and implementation

Informational · AO:S/AC:L/AX:L/C:N/I:N/A:N/D:N/Y:N/R:N/S:U

DESCRIPTION

The documentation of components touched by MIP-X71 describes behavior that the implementation does not provide in every case, which can lead to incorrect assumptions in future upgrade and governance proposals. For the internal cash accounting, the repository's contributor rules already require `allowResign = true` for every implementation upgrade of these markets, but the contract NatSpec and the governance proposal text state the guarantee unconditionally.

Instances of this issue can be found in:

- `MErc20Delegate._becomeImplementation()`: the NatSpec states that "Donations are swept by the outgoing implementation's _resignImplementation first, so a swap between internal cash implementations absorbs nothing", but the outgoing `_resignImplementation()` only runs when the admin passes `allowResign = true` to `MErc20Delegator._setImplementation()`. With `allowResign = false`, no sweep takes place and `_becomeImplementation()` sets `internalCash` to the full underlying balance, so any tokens held above `internalCash` are absorbed into the exchange rate and can no longer be removed with `_sweepExcessCash()`.
- `proposals/mips/mip-x71/x71.md`: the proposal description states that "When a market's implementation is upgraded in the future, any tokens held above its accounted cash are first transferred to the market admin, so they are not absorbed into the exchange rate", but the same `allowResign = false` path upgrades the implementation without transferring the excess. The new `MWethOwnerWrapper._setImplementation()` forwards the `allowResign` value chosen by its owner unchanged to the Base WETH market.
- `MWethOwnerWrapper`: the contract NatSpec states that the wrapper "Delegates all admin functions to the underlying WETH market", but it only forwards a fixed set of admin functions. For example, `MErc20.sweepToken()`, which only the market admin can call, has no passthrough, so tokens other than WETH sent to the Base WETH market cannot be swept through the wrapper and require a further governance action, such as moving the market admin with `_setPendingAdmin()` or upgrading the wrapper.
- `proposals/mips/mip-x71/mip-x71.sol`: the contract title states that the proposal applies "internal cash accounting for all mToken markets", but the proposal only enumerates and upgrades the markets on Ethereum, Base and OP Mainnet. The proposal description states that no Moonbeam or Moonriver markets are included, so the Moonwell markets on those networks are not upgraded to internal cash accounting.

RECOMMENDATION

Align the documentation with the implemented behavior. State in the `MErc20Delegate._becomeImplementation()` NatSpec, the `MWethOwnerWrapper._setImplementation()` NatSpec and the proposal description that tokens held above `internalCash` are swept only when `_setImplementation()` is called with `allowResign = true`, and that `allowResign = false` absorbs them into the exchange rate. Replace the `MWethOwnerWrapper` statement that it delegates all admin functions with the list of functions it forwards, and limit the `mip-x71.sol` title to the mToken markets on Ethereum, Base and OP Mainnet.

Disclaimer

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.